Comprobar resultados con JUnit

Retomemos nuestra prueba

RebalanceCalculatorTest.java comprobar-resultados-junit/after/src/test/java/…/RebalanceCalculatorTest.java

```java
// Compra dos acciones si está por debajo del objetivo.
@Test
void buysTwoSharesWhenBelowTarget() {
    double order = RebalanceCalculator.quantityToTrade(2000, 60, 100, 10);
    assertEquals(2.0, order, 0.000001);
}
```

¿Qué demuestra esta comprobación?

¿Por qué esperamos comprar 2?

Objetivo

60 % de 2000 USD = 1200 USD.

Cantidad deseada

1200 USD ÷ 100 USD = 12 acciones.

Operación

Tenemos 10. Faltan 2 acciones.

El esperado viene del requisito y de un ejemplo que podemos comprobar.

Esperado, obtenido y margen permitido

```
1  assertEquals(2.0, order, 0.000001);
```

2.0

La cantidad esperada.

order

El resultado del método.

0.000001

Diferencia absoluta permitida.

Comprar no significa comprar la cantidad correcta

RebalanceCalculatorTest.java comprobar-resultados-junit/after/src/test/java/…/RebalanceCalculatorTest.java

```
1  double order = RebalanceCalculator.quantityToTrade(2000, 60, 100, 10);
2  assertTrue(order > 0);
```

Devuelve 2

La comprobación pasa.

Devuelve 200

También pasa, aunque la cantidad es incorrecta.

Comprobemos lo que pide el requisito

```
1  assertEquals(2.0, order, 0.000001);
```

Esperamos dos acciones, no cualquier cantidad positiva.

Una prueba útil distingue el resultado correcto de un error plausible.

Un defecto que la comprobación débil acepta

RebalanceCalculator.java Defecto intencional en copia temporal; no forma parte de after.

```
1  int targetValue = totalValue * targetPercent / 10; // Debía ser 100.
2  double targetShares = targetValue / (double) price;
3  return targetShares - currentQuantity;
```

```
Tests run: 2, Failures: 1, Errors: 0

BUILD FAILURE
```

PRUEBAS 1 de 2 correctas

✓	proposesPurchaseWhenBelowTarget()	Correcta
×	buysTwoSharesWhenBelowTarget()	Falló

Leamos la falla

Qué prueba

buysTwoSharesWhenBelowTarget

Qué diferencia

Esperado: 2.0
Obtenido: 110.0

Dónde investigar

JUnit indica el archivo y la línea de la comprobación.

Revisamos requisito, datos, esperado e implementación.

Corregimos el cálculo y volvemos a ejecutar

```
1  int targetValue = totalValue * targetPercent / 100;
```

```
Tests run: 3, Failures: 0, Errors: 0

BUILD SUCCESS
```

PRUEBAS — 3 de 3 correctas

✓	calculatesFractionalQuantityToSell()	Correcta
✓	proposesPurchaseWhenBelowTarget()	Correcta
✓	buysTwoSharesWhenBelowTarget()	Correcta

No cambiamos una expectativa justificada solo para obtener verde.

Una aproximación también necesita un límite

RebalanceCalculatorTest.java comprobar-resultados-junit/after/src/test/java/…/RebalanceCalculatorTest.java

```
1  // Calcula cuántas acciones fraccionarias se deben vender.
2  double order = RebalanceCalculator.quantityToTrade(2500, 40, 150, 10);
3  assertEquals(-3.333333, order, 0.000001);
```

Diferencia permitida

−3.333333333… está suficientemente cerca de −3.333333.

Tu turno

Cuenta: 2000 USD. Objetivo: 50 %.
Precio: 100 USD. Tenemos 10 acciones.
Predice y comprueba la operación.

El margen no redondea ni representa una política financiera.